

Planificación automática

La planificación es el proceso de razonamiento explícito en el que, anticipando el resultado esperado de las posibles acciones a realizar, se seleccionan y organizan estas de tal manera que permitan alcanzar algún objetivo preestablecido.

La planificación automática es el campo de la inteligencia artificial que estudia este proceso de razonamiento de manera computacional. Por un lado, la planificación es un componente importante del comportamiento racional, por lo que desde un punto de vista científico su estudio es necesario para poder entender qué es la inteligencia. Por otro lado, desde el punto de vista de la ingeniería, el desarrollo de planificadores (es decir, programas que son capaces de planificar) es fundamental en la construcción de sistemas autónomos (un ejemplo de estos sistemas serían los robots exploradores de Marte, que no pueden ser controlados en tiempo real y, por tanto, deben ser capaces de planificar por sí mismos la mejor manera de alcanzar el objetivo establecido desde el centro de control).

Existen tres tipos principales de planificadores:

- Planificadores específicos de un dominio: están optimizados para un dominio concreto como, por ejemplo, la planificación del movimiento de un robot. Usan técnicas particulares para el problema en cuestión, que son difíciles de generalizar a otros dominios.
- Planificadores independientes del dominio: no usan ningún conocimiento específico del problema, por lo que, en principio, se pueden emplear en cualquier dominio. En la práctica, es necesario considerar simplificaciones acerca del dominio para que su utilización sea viable.
- Planificadores configurables: son planificadores independientes del dominio, pero que requieren información específica del problema como parte de los datos de entrada.

Este tema se enmarca dentro de lo que se conoce como planificación clásica, en la que, para facilitar el desarrollo de planificadores independientes del dominio, se consideran únicamente dominios que satisfacen las siguientes ocho restricciones:

1. Solo hay una cantidad finita de posibles situaciones en las que se puede encontrar el problema y solo es posible aplicar un conjunto finito de acciones (sistema finito).

2. Siempre se dispone del conocimiento completo acerca del estado en que se encuentra el problema (sistema completamente observable).
3. Siempre se obtiene el mismo resultado al aplicar la misma acción al mismo estado (sistema determinista).
4. El estado en que se encuentra el problema únicamente cambia al aplicar una acción (sistema estático).
5. El objetivo de la planificación es conseguir que el problema se encuentre en uno de los estados establecidos como objetivos (consecución de estados objetivos).
6. Una solución al problema consiste de una secuencia consecutiva de acciones (planes secuenciales).
7. La aplicación de las acciones se considera instantánea, es decir, las acciones no tienen duración (tiempo implícito).
8. Mientras se planifica una solución no se produce ningún cambio en el problema (planificación fuera de línea).

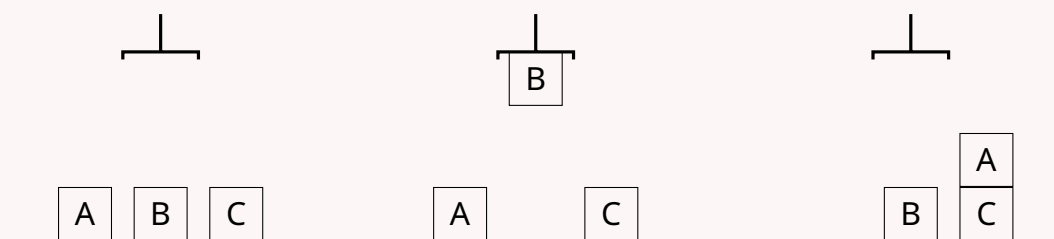
En este marco, un problema de planificación se puede entender como un sistema de transición de estados. Es decir, es una tupla $P = (S, A, s_I, S_G)$ donde

- S es un conjunto finito de estados.
- A es un conjunto finito de acciones que hacen cambiar de estado al sistema.
- s_I es el estado inicial del problema.
- S_G es el subconjunto de estados objetivo, alguno de los cuales se pretende alcanzar.

Ejemplo: El mundo de los bloques

Un dominio estándar en planificación automática es el mundo de los bloques: se tiene una mesa sobre la que hay un conjunto de bloques cúbicos; los bloques se pueden apilar, pero sobre cada bloque solo se puede colocar un bloque; se tiene también un brazo robótico que puede coger cualquier bloque, uno cada vez, y moverlo a otra posición, ya sea sobre la mesa o sobre otro bloque; el objetivo es, partiendo de una disposición inicial de los bloques, alcanzar alguna disposición específica (por ejemplo, todos los bloques apilados uno encima de otro en algún orden determinado).

Si consideramos que únicamente hay tres bloques, A, B y C, algunos posibles estados en los que podría encontrarse el problema serían:



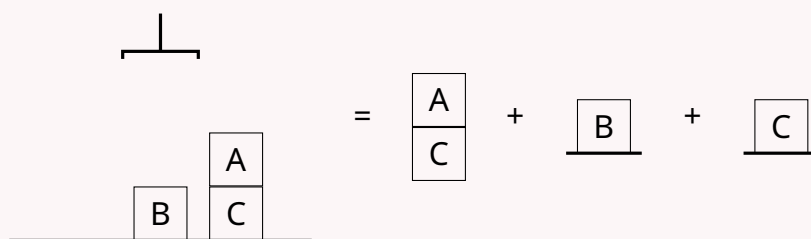
Es fácil comprobar que el problema de comenzar con una cierta disposición de los bloques y conseguir colocar estos en una de entre varias disposiciones preestablecidas, es un problema de planificación clásica: hay una cantidad finita de estados y de acciones, ya que, o bien el brazo robótico coge un bloque, o bien suelta un bloque sobre la mesa o sobre otro bloque; se conoce perfectamente toda la información que determina en qué estado se encuentra el problema, que es saber dónde se encuentra cada bloque: sobre la mesa, sobre otro bloque o cogido por el brazo robótico; el resultado de coger o soltar un bloque está completamente determinado; la disposición de los bloques solo se puede cambiar mediante las acciones del brazo robótico; el objetivo es llegar a una disposición de los bloques entre varias posibles, es decir, a ciertos estados del problema; esa disposición se alcanzará con una secuencia consecutiva de acciones del brazo robótico; no interesa el tiempo que se tarde en llevar a cabo cada acción, por lo que las podemos considerar instantáneas; además, la disposición de los bloques no cambia mientras se determina esa secuencia de acciones.

Es evidente que el mundo de los bloques es un dominio de planificación clásica desde un punto de vista abstracto. En la realidad no sería así (por ejemplo, colocar un bloque exactamente sobre otro sin que sobresalga requeriría una precisión extrema del brazo robótico), pero es una buena metodología tratar de resolver una versión simplificada del problema e ir luego incorporando las complejidades que aparecen en el mundo real.

Una característica que diferencia a los problemas de planificación automática de los sistemas de transición genéricos es el uso de una representación factorizada para los estados del problema. Esta es una representación estructurada que, en lugar de considerar los estados como un todo, los considera como una agregación de componentes simples.

Ejemplo: Representación factorizada del mundo de los bloques

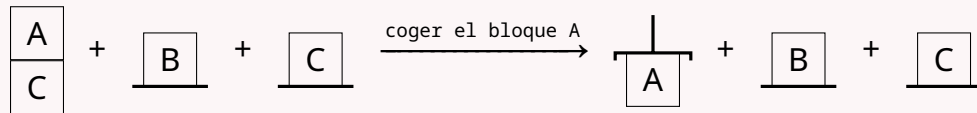
En el mundo de los bloques podemos recuperar un estado completo simplemente a partir de la posición ocupada por cada bloque, es decir, si lo ha cogido el brazo robótico o, en caso contrario, sobre qué está colocado. Por ejemplo, el siguiente estado queda completamente determinado conociendo simplemente que el bloque A está sobre el bloque C y los bloques B y C están sobre la mesa:



El uso de una representación factorizada proporciona varias ventajas:

1. Permite determinar el estado que resulta de aplicar una acción a un estado especificando únicamente qué componentes de este último cambian y cómo lo hacen. El resto de componentes permanecerán inalteradas.

Ejemplo: Aplicación de una acción en el mundo de los bloques



2. Permite detectar la necesidad de aplicar ciertas acciones antes siquiera de haber determinado qué acciones se aplicarán previamente.

Ejemplo: Acción necesaria en el mundo de los bloques

Si un estado objetivo contiene la componente , entonces puede deducirse que, para alcanzarlo, posiblemente en algún momento será necesario aplicar la regla coger el bloque A.

3. Permite definir heurísticas independientes del dominio, a partir del cómputo del número de acciones aplicables.

Representación de problemas de planificación

Para representar problemas de planificación clásica se usará el formalismo STRIPS (*STANford Research Institute Problem Solver*). En este formalismo, un problema de planificación $P = (H, A, I, G)$ viene especificado por

1. Un conjunto H finito de hechos (también llamados proposiciones), representados por símbolos de predicados aplicados a símbolos de constantes.
2. Un conjunto A finito de acciones. Para cada una de ellas es necesario proporcionar su nombre, sus precondiciones (hechos que deben cumplirse para que se pueda aplicar la acción), su lista de borrado (hechos que dejan de cumplirse tras aplicar la acción) y su lista de adición (hechos que pasan a cumplirse tras aplicar la acción), donde las dos últimas deben ser disjuntas.
3. Un estado I inicial (hechos que se cumplen inicialmente).
4. Un objetivo G (hechos que se quiere que se cumplan).

El conjunto de predicados y acciones conforman el dominio del problema. Los estados de un problema, incluido el inicial, se representan mediante conjuntos de hechos. Se asume la hipótesis del mundo cerrado, es decir, se asume que los hechos no incluidos en un estado no se cumplen. Una acción será aplicable a un estado si

este contiene todas sus precondiciones. El resultado es un estado en el que se han eliminado todos los hechos de la lista de borrado de la acción y se han añadido todos los hechos de la lista de adición. Los estados objetivos son aquellos que contienen todos y cada uno de los hechos objetivos. Un plan solución del problema será una secuencia de acciones que, al aplicarlas una detrás de otra partiendo del estado inicial, da como resultado un estado objetivo.

El mundo de los bloques

A continuación se explica el desarrollo de una posible manera de representar el dominio de planificación del mundo de los bloques.

Para determinar cada estado en que se podría encontrar un problema de este dominio necesitamos conocer dónde se encuentra cada bloque. Usaremos entonces los predicados

- **SOBRE_LA_MESA**: predicado unario que se usará para representar que un bloque está sobre la mesa.
- **SOBRE**: predicado binario que se usará para representar que un bloque está sobre otro bloque.
- **AGARRADO**: predicado unario que se usará para representar que un bloque está cogido por el brazo robótico.

De esta forma, para un problema que usara los bloques $B_1, \dots, B_n$, el conjunto de posibles hechos sería

$$\begin{aligned} &\{\text{SOBRE_LA_MESA}(b) \mid b = B_1, \dots, B_n\} \cup \\ &\{\text{SOBRE}(b_1, b_2) \mid b_1, b_2 = B_1, \dots, B_n\} \cup \\ &\{\text{AGARRADO}(b) \mid b = B_1, \dots, B_n\} \end{aligned}$$

Analizando el problema de manera abstracta, se llega a la conclusión de que las acciones posibles son que el brazo coja o que suelte un bloque. Sin embargo, el hecho de que usemos predicados distintos para representar si un bloque se encuentra sobre la mesa o sobre otro bloque nos obliga a desdoblarse las acciones en función de esas dos posibilidades. Ejemplos de acciones serían las siguientes:

- **AGARRAR(B_1)**: con esta acción el brazo robótico cogería el bloque B_1 cuando este está sobre la mesa. Esta debe ser, por tanto, una precondición de la acción. Hay dos precondiciones adicionales que deben cumplirse y que nos obligan a introducir nuevos predicados, ya que con los actuales es complejo recuperar la información que se necesita:
 - En primer lugar, puesto que el brazo robótico solo puede coger un bloque cada vez, para poder coger el bloque B_1 no debe tener ningún bloque cogido. En lugar de expresar esto usando un nuevo predicado **NO_AGARRADO** y añadiendo **NO_AGARRADO(B_2)**, ..., **NO_AGARRADO(B_n)** como precondiciones, es más simple incorporar el hecho **BRAZO_LIBRE()** a la representación.

- En segundo lugar, el brazo robótico no podrá coger el bloque B_1 si hay algún bloque encima de él. Igual que antes, en lugar de usar un nuevo predicado NO_SOBRE y añadir como precondiciones NO_SOBRE(B_2, B_1), ..., NO_SOBRE(B_n, B_1), es más simple incorporar el hecho DESPEJADO(B_1) a la representación.

El efecto de la acción sería que el bloque B_1 pasaría de estar sobre la mesa a estar agarrado por el brazo robótico, pero también hay que eliminar DESPEJADO(B_1), pues dejaría de tener sentido en la nueva situación, y BRAZO_LIBRE(), ya que ahora el brazo robótico tendría cogido un bloque.

Análogamente se definirían las acciones AGARRAR(B_2), ..., AGARRAR(B_n).

- BAJAR(B_1): con esta acción el brazo robótico dejaría el bloque B_1 sobre la mesa. La precondición para poder aplicar esta acción es, evidentemente, que el brazo robótico tenga agarrado el bloque B_1 .

Los efectos de la acción serían que el brazo robótico dejaría de tener agarrado el bloque B_1 , pasando este a estar sobre la mesa y sin ningún bloque encima.

Análogamente se definirían las acciones BAJAR(B_2), ..., BAJAR(B_n).

- DESAPILAR(B_1, B_2): con esta acción el brazo robótico cogería el bloque B_1 cuando este se encuentra sobre el bloque B_2 . Esta, junto con que sobre el bloque B_1 no haya ningún bloque y que el brazo robótico no tenga agarrado ningún bloque, serían las precondiciones.

Los efectos directos de la acción serían que el bloque B_1 pasaría de estar sobre el bloque B_2 a estar agarrado por el brazo robótico, que ya no estaría libre. Pero también el bloque B_2 pasaría a estar despejado, porque ya no tendría el bloque B_1 encima. Para este último, sin embargo, ya no tendría sentido decir que está despejado.

Análogamente se definirían las acciones DESAPILAR(B_1, B_3), ..., DESAPILAR(B_n, B_{n-1}).

- APILAR(B_1, B_2): con esta acción el brazo robótico dejaría el bloque B_1 sobre el bloque B_2 . Una precondición para poder aplicar esta acción es, evidentemente, que el brazo robótico tenga agarrado el bloque B_1 , pero también que el bloque B_2 no tenga ya otro bloque encima.

Los efectos directos de la acción serían que el brazo robótico dejaría de tener agarrado el bloque B_1 , pasando este a estar sobre el bloque B_2 y sin ningún bloque encima. El bloque B_2 dejaría de estar despejado, ya que ahora tendría el bloque B_1 encima.

Análogamente se definirían las acciones APILAR(B_1, B_3), ..., APILAR(B_n, B_{n-1}).

Obsérvese que para un problema del mundo de los bloques que use n bloques habría que definir $2n^2$ acciones en total. Es posible, sin embargo, simplificar la representación del dominio de planificación mediante el uso de esquemas de acciones: cada uno de ellos definirá un conjunto de acciones empleando variables para representar subconjuntos de objetos.

Así, podríamos representar el dominio del mundo de los bloques como sigue:

Ejemplo: Dominio del mundo de los bloques

Predicados:

- $\text{SOBRE_LA_MESA}(b)$
- $\text{SOBRE}(b_1, b_2)$
- $\text{AGARRADO}(b)$
- $\text{BRAZO_LIBRE}()$
- $\text{DESPEJADO}(b)$

Esquemas de acciones:

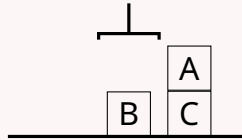
- $\text{AGARRAR}(b)$
 - Precondiciones: $\text{SOBRE_LA_MESA}(b)$
 $\text{DESPEJADO}(b)$
 $\text{BRAZO_LIBRE}()$
 - Lista de borrado: $\text{SOBRE_LA_MESA}(b)$
 $\text{DESPEJADO}(b)$
 $\text{BRAZO_LIBRE}()$
 - Lista de adición: $\text{AGARRADO}(b)$
- $\text{BAJAR}(b)$
 - Precondiciones: $\text{AGARRADO}(b)$
 - Lista de borrado: $\text{AGARRADO}(b)$
 - Lista de adición: $\text{SOBRE_LA_MESA}(b)$
 $\text{DESPEJADO}(b)$
 $\text{BRAZO_LIBRE}()$
- $\text{DESAPILAR}(b_1, b_2)$
 - Precondiciones: $\text{SOBRE}(b_1, b_2)$
 $\text{DESPEJADO}(b_1)$
 $\text{BRAZO_LIBRE}()$
 - Lista de borrado: $\text{SOBRE}(b_1, b_2)$
 $\text{DESPEJADO}(b_1)$
 $\text{BRAZO_LIBRE}()$
 - Lista de adición: $\text{AGARRADO}(b_1)$
 $\text{DESPEJADO}(b_2)$
- $\text{APILAR}(b_1, b_2)$
 - Precondiciones: $\text{AGARRADO}(b_1)$
 $\text{DESPEJADO}(b_2)$
 - Lista de borrado: $\text{AGARRADO}(b_1)$
 $\text{DESPEJADO}(b_2)$
 - Lista de adición: $\text{SOBRE}(b_1, b_2)$
 $\text{DESPEJADO}(b_1)$
 $\text{BRAZO_LIBRE}()$

Donde b, b_1 y b_2 son bloques cualesquiera.

Para definir un problema concreto en este dominio hay que especificar los bloques concretos, el estado inicial y un conjunto de estados objetivos.

Ejemplo: Estado inicial y objetivo para el mundo de los bloques

En un problema del mundo de los bloques que use los bloques A, B y C, para establecer como estado inicial el estado



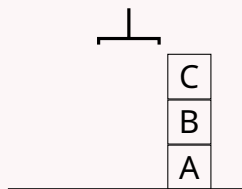
bastaría usar la siguiente representación:

SOBRE(A, C)
DESPEJADO(A)

SOBRE_LA_MESA(B)
DESPEJADO(B)

SOBRE_LA_MESA(C)
BRAZO_LIBRE()

Para establecer como único estado objetivo el estado



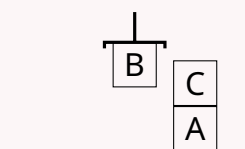
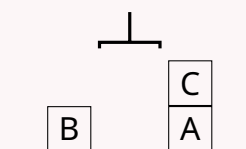
bastaría usar la siguiente representación para el objetivo:

SOBRE_LA_MESA(A)

SOBRE(B, A)

SOBRE(C, B)

Por el contrario, para establecer como estados objetivos los dos estados



se podría usar la siguiente representación para el objetivo:

SOBRE_LA_MESA(A)

SOBRE(C, A)

DESPEJADO(C)

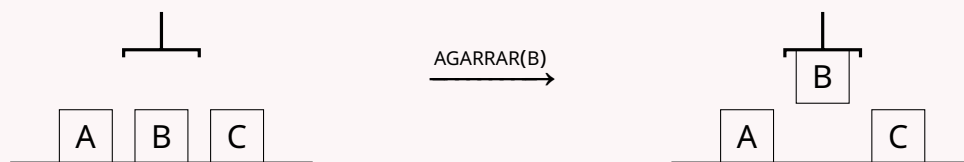
PDDL (*Planning Domain Definition Language*) es un lenguaje que implementa en la práctica el formalismo STRIPS, especificando una sintaxis estricta para la escritura de problemas de planificación en ficheros de texto llano. Este lenguaje se ha convertido en el formato de entrada *de facto* usado por todas las implementaciones de planificadores. PDDL ha evolucionado a lo largo de su historia, ampliando STRIPS con diversas extensiones, como la posibilidad de usar precondiciones negadas y cuantificadas, efectos condicionales, variables numéricas, etc.

Resolución de problemas de planificación

Una posible manera de abordar la resolución de un problema de planificación clásica es mediante una búsqueda en el espacio de estados del problema. Esta consiste en aplicar un algoritmo de búsqueda de caminos en el grafo dirigido cuyos vértices son los posibles estados del problema y cuyos arcos conectan estados para los que existe una acción que transforma uno de ellos en el otro. Una plan solución del problema vendría dado por cualquier camino desde el estado inicial hasta algún estado objetivo.

Ejemplo: Grafo del mundo de los bloques

En el mundo de los bloques con tres bloques A, B y C habría que buscar, en un grafo con 22 vértices (uno por cada posible estado del problema), un camino desde el estado que representa la disposición inicial de los bloques hasta un estado que represente alguna de las disposiciones finales deseadas. En ese grafo, dos estados estarían conectados si alguna acción del brazo robótico transformara uno en otro. Por ejemplo, en ese grafo nos encontraríamos el arco:



Algoritmos de búsqueda de caminos en grafos

Es habitual que, para resolver un problema de planificación, los algoritmos de búsqueda construyan el grafo asociado de manera implícita. El esquema general es disponer de una estructura de nodos cerrados, que son aquellos nodos del grafo ya expandidos, y una estructura de nodos abiertos, que son aquellos nodos del grafo ya generados, pero aún no expandidos. Comenzando por el nodo correspondiente al estado inicial, el comportamiento genérico de estos algoritmos consiste en iterar el proceso de seleccionar un nodo abierto, expandirlo generando sus nodos sucesores y cerrarlo una vez expandido, construyendo lo que se conoce como árbol de búsqueda. Las distintas maneras de seleccionar el nodo abierto y, al cerrar este, actualizar las estructuras de nodos abiertos y cerrados, dan lugar a distintos algoritmos de búsqueda.

Cuando la estructura de nodos abiertos es una pila (es decir, una estructura LIFO —*Last In, First Out*—, en la que se selecciona el último nodo introducido) se obtiene un algoritmo de búsqueda en profundidad. Cuando la estructura de nodos abiertos es una cola (es decir, una estructura FIFO —*First In, First Out*—, en la que se selecciona el primer nodo introducido) se obtiene un algoritmo de búsqueda en anchura.

Cuando cada acción a del problema de planificación (y, por tanto, los arcos del grafo asociado) tiene asociado un coste c_a (no negativo), el algoritmo de Dijkstra calcula un plan óptimo, es decir, un camino en el grafo desde el nodo inicial hasta un nodo objetivo con el menor coste posible. Este algoritmo se obtiene usando como estructura de nodos abiertos una cola de prioridades en la que a cada nodo s se le asocia un valor $f(s)$ que acota superiormente el mínimo coste de un camino desde el nodo inicial s_1 hasta s y que es disminuido sucesivamente hasta que coincide con ese coste mínimo.

ALGORITMO DE BÚSQUEDA EN PROFUNDIDAD/ANCHURA

ENTRADA: problema de planificación $P = (H, A, I, G)$

SALIDA: un plan solución de P , si existe

```

1  Cerrados  $\leftarrow \emptyset$ 
2  Abiertos  $\leftarrow \{I\}$ 
3  Mientras Abiertos  $\neq \emptyset$  hacer
4      (Búsqueda en profundidad) Extraer el último nodo  $s$  de Abiertos
      (Búsqueda en anchura) Extraer el primer nodo  $s$  de Abiertos
5      Insertar  $s$  en Cerrados
6      Si  $G \subseteq s$  entonces
7          Devolver el camino (plan solución de  $P$ ) desde  $I$  hasta  $s$ 
8      Para cada acción  $a \in A$  aplicable a  $s$  hacer
9           $s' \leftarrow$  sucesor de  $s$  obtenido al aplicar  $a$ 
10         Si  $s'$  no está ni en Abiertos ni en Cerrados entonces
11             Padre( $s'$ )  $\leftarrow s$ 
12             Insertar  $s'$  en Abiertos
13 Devolver no existe ningún plan solución de  $P$ 

```

Una función heurística es una función que para cada estado del problema estima el coste de un plan óptimo desde ese estado hasta el objetivo. El algoritmo A^* es capaz de incorporar esa información para guiar la búsqueda, haciéndola más eficiente. Este algoritmo se obtiene usando como estructura de nodos abiertos una cola de prioridades en la que a cada nodo s se le asocia el valor $g(s) + h(s)$, donde $g(s)$ es el valor (conocido) del camino óptimo actual desde el nodo inicial s_1 hasta s y $h(s)$ es el coste mínimo (estimado) desde s hasta el objetivo.

Una función heurística h es:

- Segura, si $h(s) = +\infty$ implica que no hay camino desde s hasta el objetivo. Es decir, para cualquier estado, si asume que es el estado inicial y estima que el problema no tiene solución, entonces no está equivocada.
- Consciente del objetivo, si $h(s) = 0$ para cualquier estado tal que $G \subseteq s$. Es decir, no estima un coste positivo de alcanzar el objetivo desde un estado objetivo.
- Admisible, si para cualquier estado el coste mínimo estimado desde ese estado hasta el objetivo es menor o igual que el coste mínimo real. Es decir, siempre subestima el coste real de llegar hasta el objetivo.
- Consistente, si para cualesquiera estado s , acción a aplicable a s y estado resultante s' se tiene que $h(s) \leq c_a + h(s')$. Es decir, al aplicar una acción, su estimación de llegar al objetivo no disminuye en mayor medida que el coste de aplicar la acción.

Una heurística admisible es segura y consciente del objetivo. Una heurística consistente y consciente del objetivo es admisible.

Usando una heurística admisible el algoritmo A^* es completo (si el problema tiene solución, entonces el algoritmo encuentra un plan solución) y óptimo (si el problema tiene solución, entonces el plan proporcionado por el algoritmo tiene coste mínimo).

ALGORITMO DE DIJKSTRA

ENTRADA: problema de planificación $P = (H, A, I, G)$

SALIDA: un plan solución de P , si existe

```

1   $f(s) \leftarrow \begin{cases} 0 & \text{para } s = I \\ +\infty & \text{para } s \neq I \end{cases}$ 
2  Cerrados  $\leftarrow \emptyset$ 
3  Abiertos  $\leftarrow \{I\}$ 
4  Mientras Abiertos  $\neq \emptyset$  hacer
5      Extraer el nodo  $s$  de Abiertos con menor valor  $f(s)$ 
6      Insertar  $s$  en Cerrados
7      Si  $G \subseteq s$  entonces
8          Devolver el camino (plan solución de  $P$ ) desde  $I$  hasta  $s$ 
9      Para cada acción  $a \in A$  aplicable a  $s$  hacer
10          $s' \leftarrow$  sucesor de  $s$  obtenido al aplicar  $a$ 
11         Si  $s'$  está en Abiertos y  $f(s) + c_a < f(s')$  entonces
12             Padre( $s'$ )  $\leftarrow s$ 
13              $f(s') \leftarrow f(s) + c_a$ 
14         Si no si  $s'$  no está en Abiertos ni en Cerrados entonces
15             Padre( $s'$ )  $\leftarrow s$ 
16              $f(s') \leftarrow f(s) + c_a$ 
17             Insertar  $s'$  en Abiertos
18 Devolver no existe ningún plan solución de  $P$ 

```

Heurísticas independientes del dominio

La cantidad de estados de un problema de planificación clásica es, en general, demasiado elevado para que se pueda resolver mediante una búsqueda ciega, como serían una búsqueda en anchura o profundidad.

Ejemplo: Cantidad de posibles estados en el mundo de los bloques

La siguiente tabla muestra el crecimiento exponencial de la cantidad de posibles estados en el mundo de los bloques en función del número de bloques:

Bloques	Estados
1	2
2	5
3	22
4	125
5	866
6	7057
7	65 990
8	695 417
9	8 145 730

ALGORITMO A*

ENTRADA: problema de planificación $P = (H, A, I, G)$

SALIDA: un plan solución de P , si existe

```

1   $g(s) \leftarrow \begin{cases} 0 & \text{para } s = I \\ +\infty & \text{para } s \neq I \end{cases}$ 
2   $f(s) \leftarrow \begin{cases} h(s) & \text{para } s = I \\ +\infty & \text{para } s \neq I \end{cases}$ 
3  Cerrados  $\leftarrow \emptyset$ 
4  Abiertos  $\leftarrow \{I\}$ 
5  Mientras Abiertos  $\neq \emptyset$  hacer
6      Extraer el nodo  $s$  de Abiertos con menor valor  $f(s)$ 
7      Insertar  $s$  en Cerrados
8      Si  $G \subseteq s$  entonces
9          Devolver el camino (plan solución de  $P$ ) desde  $I$  hasta  $s$ 
10     Para cada acción  $a \in A$  aplicable a  $s$  hacer
11          $s' \leftarrow$  sucesor de  $s$  obtenido al aplicar  $a$ 
12         Si  $s'$  está en Abiertos y  $g(s) + c_a < g(s')$  entonces
13             Padre( $s'$ )  $\leftarrow s$ 
14              $g(s') \leftarrow g(s) + c_a$ 
15              $f(s') \leftarrow g(s') + h(s')$ 
16         Si no si  $s'$  está en Cerrados y  $g(s) + c_a < g(s')$  entonces
17             Padre( $s'$ )  $\leftarrow s$ 
18              $g(s') \leftarrow g(s) + c_a$ 
19              $f(s') \leftarrow g(s') + h(s')$ 
20             Eliminar  $s'$  de Cerrados
21             Insertar  $s'$  en Abiertos
22         Si no si  $s'$  no está en Abiertos ni en Cerrados entonces
23             Padre( $s'$ )  $\leftarrow s$ 
24              $g(s') \leftarrow g(s) + c_a$ 
25              $f(s') \leftarrow g(s') + h(s')$ 
26             Insertar  $s'$  en Abiertos
27 Devolver no existe ningún plan solución de  $P$ 

```

Es necesario recurrir entonces a algoritmos como A*, en los que se guía la búsqueda mediante alguna función heurística. Una vía para definir estas funciones heurísticas de forma «automática» es mediante el uso de la técnica de relajación del problema, que consiste en eliminar restricciones del problema de tal manera que sea entonces posible determinar el coste exacto de una solución óptima.

Una manera habitual de aplicar esa técnica a los problemas de planificación es la relajación del borrado: se descartan las listas de borrado de las acciones, asumiendo entonces que su ejecución solo puede dar lugar a que se cumplan nuevos hechos, pero no a que haya hechos que dejen de cumplirse. Se elimina así la restricción de que en un estado no puedan cumplirse simultáneamente hechos incompatibles (como, por ejemplo, que el brazo robótico tenga agarrados más de un bloque a la vez).

Dado un problema de planificación $P = (H, A, I, G)$:

- Si $a \in A$ es una acción con precondiciones $\text{pre}(a)$, lista de borrado $\text{del}(a)$, lista de adición $\text{add}(a)$ y, en su caso, coste c_a , entonces la correspondiente acción relajada es la acción a^+ con precondiciones $\text{pre}(a)$, lista de borrado vacía, lista de adición $\text{add}(a)$ y coste c_a .
- El problema de planificación relajado correspondiente a P es $P^+ = (H, A^+, I, G)$, donde A^+ es el conjunto de las acciones relajadas obtenidas a partir de todas las acciones en A .
- Si s es un estado del problema, entonces $P_s = (H, A, s, G)$ es el problema que se obtiene a partir de P estableciendo s como estado inicial. Un plan relajado para s es un plan solución de P_s^+ . Un plan relajado para P es un plan relajado para I .
- Si $\langle a_1, \dots, a_n \rangle$ es un plan solución de P_s , entonces $\langle a_1^+, \dots, a_n^+ \rangle$ es un plan relajado para s . Por lo tanto, si P_s es resoluble, entonces P_s^+ también lo es.

Ejemplo: Plan relajado para el problema del transporte de paquetes

Consideremos el dominio de planificación consistente en una serie de lugares, camiones y paquetes de tal forma que: algunos de los lugares están conectados entre sí; los camiones se pueden mover entre dos lugares si estos están conectados; un camión puede cargar un paquete si ambos se hallan en el mismo lugar (asumimos que los camiones tienen capacidad infinita); si un camión tiene cargado un paquete, entonces lo puede descargar en el lugar en el que se halle (asumimos que los lugares tienen capacidad infinita).

Dentro de este dominio, consideremos el problema formado por cuatro lugares L_0, L_1, L_2 y L_3 , un único camión C y un único paquete P . Los lugares L_i y L_{i+1} , para $i = 0, 1, 2$, y los lugares L_1 y L_3 están conectados entre sí. El camión C se halla inicialmente en el lugar L_0 y el paquete P en el lugar L_1 . El objetivo es que el camión lleve el paquete al lugar L_3 y vuelva al lugar L_0 .

Una posible representación del problema usando el formalismo STRIPS sería:

Predicados:

- $\text{CONECTADOS}(l_1, l_2)$, con l_1 y l_2 lugares.
- $\text{CAMIÓN_EN}(c, l)$, con c camión y l lugar.
- $\text{PAQUETE_EN}(p, lc)$, con p paquete y lc lugar o camión.

Esquemas de acciones:

- $\text{IR}(c, l_1, l_2)$, con c camión, l_1 y l_2 lugares
 - Precondiciones: $\text{CONECTADOS}(l_1, l_2)$
 $\text{CAMIÓN_EN}(c, l_1)$
 - Lista de borrado: $\text{CAMIÓN_EN}(c, l_1)$
 - Lista de adición: $\text{CAMIÓN_EN}(c, l_2)$

- CARGAR(p, c, l), con p paquete, c camión y l lugar
 - Precondiciones: CAMIÓN_EN(c, l)
PAQUETE_EN(p, l)
 - Lista de borrado: PAQUETE_EN(p, l)
 - Lista de adición: PAQUETE_EN(p, c)
- DESCARGAR(p, c, l), con p paquete, c camión y l lugar
 - Precondiciones: CAMIÓN_EN(c, l)
PAQUETE_EN(p, c)
 - Lista de borrado: PAQUETE_EN(p, c)
 - Lista de adición: PAQUETE_EN(p, l)

Estado inicial: CONECTADOS(L_0, L_1) CONECTADOS(L_2, L_3) CAMIÓN_EN(C, L_0)
 CONECTADOS(L_1, L_0) CONECTADOS(L_3, L_2) PAQUETE_EN(P, L_1)
 CONECTADOS(L_1, L_2) CONECTADOS(L_1, L_3)
 CONECTADOS(L_2, L_1) CONECTADOS(L_3, L_1)

Nota: los hechos que especifican los lugares conectados son fijos y en lo que sigue se asumen incluidos en todos los estados.

Objetivo: CAMIÓN_EN(C, L_0)
 PAQUETE_EN(P, L_3)

A continuación usamos el método de relajación del borrado.

Esquemas de acciones relajadas:

- IR(c, l_1, l_2)⁺, con c camión, l_1 y l_2 lugares
 - Precondiciones: CONECTADOS(l_1, l_2)
CAMIÓN_EN(c, l_1)
 - Lista de adición: CAMIÓN_EN(c, l_2)
- CARGAR(p, c, l)⁺, con p paquete, c camión y l lugar
 - Precondiciones: CAMIÓN_EN(c, l)
PAQUETE_EN(p, l)
 - Lista de adición: PAQUETE_EN(p, c)
- DESCARGAR(p, c, l)⁺, con p paquete, c camión y l lugar
 - Precondiciones: CAMIÓN_EN(c, l)
PAQUETE_EN(p, c)
 - Lista de adición: PAQUETE_EN(p, l)

Un *plan relajado* para el problema sería

$\langle \text{IR}(C, L_0, L_1)^+, \text{CARGAR}(P, C, L_1)^+, \text{IR}(C, L_1, L_3)^+, \text{DESCARGAR}(P, C, L_3)^+ \rangle$

que daría lugar a la siguiente secuencia de estados del problema relajado:

- Estado inicial:

$\text{CAMIÓN_EN}(C, L_0)$
 $\text{PAQUETE_EN}(P, L_1)$

- Tras aplicar la acción $\text{IR}(C, L_0, L_1)^+$:

$\text{CAMIÓN_EN}(C, L_0)$ $\text{CAMIÓN_EN}(C, L_1)$
 $\text{PAQUETE_EN}(P, L_1)$

- Tras aplicar la acción $\text{CARGAR}(P, C, L_1)^+$:

$\text{CAMIÓN_EN}(C, L_0)$ $\text{CAMIÓN_EN}(C, L_1)$
 $\text{PAQUETE_EN}(P, L_1)$ $\text{PAQUETE_EN}(P, C)$

- Tras aplicar la acción $\text{IR}(C, L_1, L_3)^+$:

$\text{CAMIÓN_EN}(C, L_0)$ $\text{CAMIÓN_EN}(C, L_1)$ $\text{CAMIÓN_EN}(C, L_3)$
 $\text{PAQUETE_EN}(P, L_1)$ $\text{PAQUETE_EN}(P, C)$

- Tras aplicar la acción $\text{DESCARGAR}(P, C, L_3)^+$:

$\text{CAMIÓN_EN}(C, L_0)$ $\text{CAMIÓN_EN}(C, L_1)$ $\text{CAMIÓN_EN}(C, L_3)$
 $\text{PAQUETE_EN}(P, L_1)$ $\text{PAQUETE_EN}(P, C)$ $\text{PAQUETE_EN}(P, L_3)$

Obsérvese cómo en el problema relajado no es necesario aplicar acciones para que el camión vuelva desde L_3 hasta L_0 , ya que los estados conservan la información de todas las localizaciones por las que ha pasado.

Un plan relajado para s resuelve el problema P_s partiendo del estado s y simplemente añadiendo cada vez nuevos hechos que se cumplen, sin tener en cuenta las posibles interacciones entre las acciones. Esta consideración nos lleva al algoritmo voraz de cálculo de planes relajados para s .

Ejemplo: Cálculo de un plan relajado para el problema del transporte de paquetes

A continuación simulamos la ejecución del algoritmo voraz de cálculo de planes relajados para obtener un plan relajado, de entre todos los posibles, para el problema del camión y el paquete.

ALGORITMO VORAZ DE CÁLCULO DE PLANES RELAJADOS

ENTRADA: problema de planificación $P = (H, A, I, G)$ y estado s de P

SALIDA: un plan relajado para s , si existe

```

1   $s^+ \leftarrow s$ 
2   $a^+ \leftarrow \langle \rangle$ 
3  Mientras  $G \not\subseteq s^+$  hacer
4      Si existe  $a \in A$  tal que  $\text{pre}(a) \subseteq s^+$  y  $\text{add}(a) \not\subseteq s^+$  entonces
5          Elegir una tal acción  $a$ 
6          Añadir  $\text{add}(a)$  a  $s^+$ 
7          Añadir  $a^+$  al final de  $a^+$ 
8      Si no entonces
9          Devolver no existe plan relajado para  $s$ 
10 Devolver  $a^+$ 

```

- Inicialización (recuérdese que los hechos que especifican los lugares conectados se asumen incluidos en todos los estados):

$$s^+ \leftarrow \text{CAMIÓN_EN}(C, L_0) \quad \text{PAQUETE_EN}(P, L_1)$$

$$a^+ \leftarrow \langle \rangle$$

- Acciones elegibles: $\text{IR}(C, L_0, L_1)$

Se elige la acción $\text{IR}(C, L_0, L_1)$:

$$s^+ \leftarrow \text{CAMIÓN_EN}(C, L_0) \quad \text{CAMIÓN_EN}(C, L_1)$$

$$\text{PAQUETE_EN}(P, L_1)$$

$$a^+ \leftarrow \langle \text{IR}(C, L_0, L_1)^+ \rangle$$

- Acciones elegibles: $\text{IR}(C, L_1, L_2)$, $\text{IR}(C, L_1, L_3)$, $\text{CARGAR}(P, C, L_1)$
(las acciones $\text{IR}(C, L_0, L_1)$ e $\text{IR}(C, L_1, L_0)$ también son aplicables, pero no proporcionan hechos nuevos)

Se elige la acción $\text{CARGAR}(P, C, L_1)$:

$$s^+ \leftarrow \text{CAMIÓN_EN}(C, L_0) \quad \text{CAMIÓN_EN}(C, L_1)$$

$$\text{PAQUETE_EN}(P, L_1) \quad \text{PAQUETE_EN}(P, C)$$

$$a^+ \leftarrow \langle \text{IR}(C, L_0, L_1)^+, \text{CARGAR}(P, C, L_1)^+ \rangle$$

- Acciones elegibles: $\text{IR}(C, L_1, L_2)$, $\text{IR}(C, L_1, L_3)$, $\text{DESCARGAR}(P, C, L_0)$

Se elige la acción $\text{DESCARGAR}(P, C, L_0)$:

$$s^+ \leftarrow \text{CAMIÓN_EN}(C, L_0) \quad \text{CAMIÓN_EN}(C, L_1)$$

$$\text{PAQUETE_EN}(P, L_1) \quad \text{PAQUETE_EN}(P, C) \quad \text{PAQUETE_EN}(P, L_0)$$

$$a^+ \leftarrow \langle \text{IR}(C, L_0, L_1)^+, \text{CARGAR}(P, C, L_1)^+, \text{DESCARGAR}(P, C, L_0)^+ \rangle$$

- Acciones elegibles: $IR(C, L_1, L_2)$, $IR(C, L_1, L_3)$

Se elige la acción $IR(C, L_1, L_3)$:

$$s^+ \leftarrow \begin{array}{l} CAMIÓN_EN(C, L_0) \quad CAMIÓN_EN(C, L_1) \quad CAMIÓN_EN(C, L_3) \\ PAQUETE_EN(P, L_1) \quad PAQUETE_EN(P, C) \quad PAQUETE_EN(P, L_0) \end{array}$$

$$a^+ \leftarrow \langle IR(C, L_0, L_1)^+, CARGAR(P, C, L_1)^+, DESCARGAR(P, C, L_0)^+, IR(C, L_1, L_3)^+ \rangle$$

- Acciones elegibles: $IR(C, L_1, L_2)$, $IR(C, L_3, L_2)$, $DESCARGAR(P, C, L_3)$

Se elige la acción $DESCARGAR(P, C, L_3)$:

$$s^+ \leftarrow \begin{array}{l} CAMIÓN_EN(C, L_0) \quad CAMIÓN_EN(C, L_1) \quad CAMIÓN_EN(C, L_3) \\ PAQUETE_EN(P, L_1) \quad PAQUETE_EN(P, C) \quad PAQUETE_EN(P, L_0) \\ PAQUETE_EN(P, L_3) \end{array}$$

$$a^+ \leftarrow \langle IR(C, L_0, L_1)^+, CARGAR(P, C, L_1)^+, DESCARGAR(P, C, L_0)^+, IR(C, L_1, L_3)^+, \\ DESCARGAR(P, C, L_3)^+ \rangle$$

En este punto, el estado s^+ satisface el objetivo del problema, por lo que el algoritmo termina y devuelve

$$\langle IR(C, L_0, L_1)^+, CARGAR(P, C, L_1)^+, DESCARGAR(P, C, L_0)^+, IR(C, L_1, L_3)^+, \\ DESCARGAR(P, C, L_3)^+ \rangle$$

como plan relajado del problema.

Se podría considerar entonces

$$h(s) = \begin{cases} \sum_{i=1}^n c_a & \text{si } \langle a_1^+, \dots, a_n^+ \rangle \text{ es un plan relajado para } s, \\ +\infty & \text{si no existe plan relajado para } s. \end{cases}$$

como estimación del coste mínimo de alcanzar el objetivo del problema de planificación desde cada posible estado s . Sin embargo, esta no sería una definición adecuada para la heurística por las siguientes dos razones:

- El algoritmo de obtención de un plan relajado para s puede seleccionar acciones irrelevantes (como la acción $DESCARGAR(P, C, L_0)$ en el ejemplo anterior), sobreestimando de esta forma el coste necesario para alcanzar el objetivo.
- La heurística puede proporcionar estimaciones muy distintas para estados similares, e incluso al calcularla repetidamente para un mismo estado.

Es por ello que la función heurística que realmente debe considerarse es

$$h^+(s) = \begin{cases} \sum_{i=1}^n c_a & \text{si } \langle a_1^+, \dots, a_n^+ \rangle \text{ es un plan relajado óptimo para } s, \\ +\infty & \text{si no existe plan relajado para } s. \end{cases}$$

donde un plan relajado óptimo para s es un plan relajado para s de coste mínimo.

Para calcular $h^+(s)$ basta, por tanto, obtener todos los posibles planes relajados para s (para ello habría que aplicar el algoritmo voraz eligiendo las acciones de todas las maneras posibles), medir sus costes (es decir, la suma de los costes de las acciones que contienen) y seleccionar el menor de entre todos ellos.

h^+ es una heurística segura, consciente del objetivo y consistente (y, por tanto, admisible), pero su cálculo es computacionalmente costoso, ya que la cantidad de posibles planes relajados para un estado es, en general, exponencial en el número de acciones del dominio. Esto hace que, en la práctica, se consideren funciones heurísticas que aproximen h^+ de una u otra manera, pero que sean más eficientes de calcular.

Una de las maneras más habituales de realizar esa aproximación es considerar independencia entre los objetivos a alcanzar. Por ejemplo, la heurística $h^{\max}$ asume que para lograr un conjunto de objetivos es suficiente con lograr el objetivo más costoso, lo que da lugar a la siguiente definición recursiva:

$$h^{\max}(s, G) = \begin{cases} 0 & \text{si } G = \{g\} \text{ y } g \in s, \\ \min(h^{\max}(s, \text{pre}(a)) + c_a \mid a \in A, g \in \text{add}(a)) & \text{si } G = \{g\} \text{ y } g \notin s, \\ \max(h^{\max}(s, \{g\}) \mid g \in G) & \text{si } |G| > 1. \end{cases}$$

$$h^{\max}(s) = h^{\max}(s, S_G)$$

Por su parte, la heurística h^{add} asume que para lograr un conjunto de objetivos se debe lograr cada uno de los objetivos por separado, lo que da lugar a la siguiente definición recursiva:

$$h^{\text{add}}(s, G) = \begin{cases} 0 & \text{si } G = \{g\} \text{ y } g \in s, \\ \min(h^{\text{add}}(s, \text{pre}(a)) + c_a \mid a \in A, g \in \text{add}(a)) & \text{si } G = \{g\} \text{ y } g \notin s, \\ \sum_{g \in G} h^{\text{add}}(s, \{g\}) & \text{si } |G| > 1. \end{cases}$$

$$h^{\text{add}}(s) = h^{\text{add}}(s, S_G)$$

Aplicar directamente las definiciones recursivas anteriores para el cálculo de $h^{\max}$ y h^{add} no es adecuado, porque pueden haber dependencias cíclicas. Lo correcto es usar en su lugar un algoritmo de programación dinámica.

Ejemplo: Cálculo de $h^{\max}(I)$ para el problema del transporte de paquetes

Asumiendo que las acciones $\text{IR}(C, L_1, L_3)$ y $\text{IR}(C, L_3, L_1)$ tienen coste 3, el resto de acciones IR tienen coste 1 y todas las acciones CARGAR y DESCARGAR tienen coste 0, se pretende calcular el valor de $h^{\max}$ para el estado inicial I del problema.

La siguiente tabla muestra los valores de las distintas funciones T_i^I calculadas por el algoritmo. Estas funciones se pueden entender como distintas iteraciones de una única función T^I cuyos valores para cada posible hecho han sido finalmente determinados cuando coinciden dos iteraciones consecutivas.

ALGORITMO DE CÁLCULO DE $h^{\max}/h^{\text{add}}$

ENTRADA: problema de planificación $P = (H, A, I, G)$ y estado s de P

SALIDA: $h^{\max}(s)$

```

1  Para cada hecho  $g \in H$  hacer
2      Inicializar  $T_0^s(g)$  como  $\begin{cases} 0 & \text{si } g \in s, \\ \infty & \text{si } g \notin s. \end{cases}$ 
3   $i \leftarrow 0$ 
4  Repetir
5       $i \leftarrow i + 1$ 
6      Para cada hecho  $g \in H$  hacer
7          Para cada acción  $a \in A$  tal que  $g \in \text{add}(a)$  hacer
8               $(h^{\max}) c_{\text{pre}(a)} \leftarrow \max(T_{i-1}^s(g') \mid g' \in \text{pre}(a))$ 
8               $(h^{\text{add}}) c_{\text{pre}(a)} \leftarrow \sum_{g' \in \text{pre}(a)} T_{i-1}^s(g')$ 
9               $c_g \leftarrow \min(c_{\text{pre}(a)} + c_a \mid a \in A, g \in \text{add}(a))$ 
10              $T_i^s(g) \leftarrow \min(T_{i-1}^s(g), c_g)$ 
11         Si  $T_i^s = T_{i-1}^s$  entonces
12              $(h^{\max})$  Devolver  $\max(T_i^s(g) \mid g \in G)$ 
12              $(h^{\text{add}})$  Devolver  $\sum_{g \in G} T_i^s(g)$ 

```

Hecho	T_0^I	T_1^I	T_2^I	T_3^I	T_4^I	T_5^I
CAMIÓN_EN(C, L ₀)	0	0	0	0	0	0
CAMIÓN_EN(C, L ₁)	∞	1	1	1	1	1
CAMIÓN_EN(C, L ₂)	∞	∞	2	2	2	2
CAMIÓN_EN(C, L ₃)	∞	∞	4	3	3	3
PAQUETE_EN(P, L ₀)	∞	∞	∞	1	1	1
PAQUETE_EN(P, L ₁)	0	0	0	0	0	0
PAQUETE_EN(P, L ₂)	∞	∞	∞	2	2	2
PAQUETE_EN(P, L ₃)	∞	∞	∞	4	3	3
PAQUETE_EN(P, C)	∞	∞	1	1	1	1
CONECTADOS(L _i , L _j) si L _i y L _j están conectados	0	0	0	0	0	0
CONECTADOS(L _i , L _j) si L _i y L _j no están conectados	∞	∞	∞	∞	∞	∞

Por ejemplo, para calcular el valor de T_1^I para el hecho CAMIÓN_EN(C, L₁) el algoritmo:

- Determina que las acciones que tienen a CAMIÓN_EN(C, L₁) en la lista de adición son IR(C, L₀, L₁), IR(C, L₁, L₁), IR(C, L₂, L₁) e IR(C, L₃, L₁).

- Las precondiciones de $IR(C, L_0, L_1)$ son $CONECTADOS(L_0, L_1)$ y $CAMIÓN_EN(C, L_0)$, luego calcula

$$\begin{aligned} c_{pre(IR(C, L_0, L_1))} &\leftarrow \max(T_0^I(CONECTADOS(L_0, L_1)), T_0^I(CAMIÓN_EN(C, L_0))) \\ &= \max(0, 0) \\ &= 0 \end{aligned}$$

- Las precondiciones de $IR(C, L_1, L_1)$ son $CONECTADOS(L_1, L_1)$ y $CAMIÓN_EN(C, L_1)$, luego calcula

$$\begin{aligned} c_{pre(IR(C, L_1, L_1))} &\leftarrow \max(T_0^I(CONECTADOS(L_1, L_1)), T_0^I(CAMIÓN_EN(C, L_1))) \\ &= \max(\infty, \infty) \\ &= \infty \end{aligned}$$

- Las precondiciones de $IR(C, L_2, L_1)$ son $CONECTADOS(L_2, L_1)$ y $CAMIÓN_EN(C, L_2)$, luego calcula

$$\begin{aligned} c_{pre(IR(C, L_2, L_1))} &\leftarrow \max(T_0^I(CONECTADOS(L_2, L_1)), T_0^I(CAMIÓN_EN(C, L_2))) \\ &= \max(0, \infty) \\ &= \infty \end{aligned}$$

- Las precondiciones de $IR(C, L_3, L_1)$ son $CONECTADOS(L_3, L_1)$ y $CAMIÓN_EN(C, L_3)$, luego calcula

$$\begin{aligned} c_{pre(IR(C, L_3, L_1))} &\leftarrow \max(T_0^I(CONECTADOS(L_3, L_1)), T_0^I(CAMIÓN_EN(C, L_3))) \\ &= \max(0, \infty) \\ &= \infty \end{aligned}$$

- Calcula $c_{CAMIÓN_EN(C, L_1)} \leftarrow \min(c_{pre(IR(C, L_0, L_1))} + c_{IR(C, L_0, L_1)},$
 $c_{pre(IR(C, L_1, L_1))} + c_{IR(C, L_1, L_1)},$
 $c_{pre(IR(C, L_2, L_1))} + c_{IR(C, L_2, L_1)},$
 $c_{pre(IR(C, L_3, L_1))} + c_{IR(C, L_3, L_1)})$
 $= \min(0 + 1, \infty + 1, \infty + 1, \infty + 3)$
 $= 1$

- Establece $T_1^I(CAMIÓN_EN(C, L_1)) \leftarrow \min(T_0^I(CAMIÓN_EN(C, L_1)), c_{CAMIÓN_EN(C, L_1)})$
 $= \min(\infty, 1)$
 $= 1$

Finalmente, una vez la función T^I es estable, se tiene que

$$\begin{aligned} h^{\max}(I) &= \max(T^I(CAMIÓN_EN(C, L_0)), T^I(PAQUETE_EN(P, L_3))) \\ &= \max(0, 3) \\ &= 3 \end{aligned}$$

$h^{\max}$ es una heurística segura, consciente del objetivo y admisible, ya que $h^{\max} \leq h^*$. Puede, sin embargo, subestimar en demasía el coste real de alcanzar el objetivo.

Ejemplo: Cálculo de $h^{\text{add}}(I)$ para el problema del transporte de paquetes

Asumiendo que las acciones $\text{IR}(C, L_1, L_3)$ y $\text{IR}(C, L_3, L_1)$ tienen coste 3, el resto de acciones IR tienen coste 1 y todas las acciones CARGAR y DESCARGAR tienen coste 0, se pretende calcular el valor de h^{add} para el estado inicial I del problema.

La siguiente tabla muestra los valores de las distintas funciones T_i^I calculadas por el algoritmo. Estas funciones se pueden entender como distintas iteraciones de una única función T^I cuyos valores para cada posible hecho han sido finalmente determinados cuando coinciden dos iteraciones consecutivas.

Hecho	T_0^I	T_1^I	T_2^I	T_3^I	T_4^I	T_5^I
$\text{CAMIÓN_EN}(C, L_0)$	0	0	0	0	0	0
$\text{CAMIÓN_EN}(C, L_1)$	∞	1	1	1	1	1
$\text{CAMIÓN_EN}(C, L_2)$	∞	∞	2	2	2	2
$\text{CAMIÓN_EN}(C, L_3)$	∞	∞	4	3	3	3
$\text{PAQUETE_EN}(P, L_0)$	∞	∞	∞	1	1	1
$\text{PAQUETE_EN}(P, L_1)$	0	0	0	0	0	0
$\text{PAQUETE_EN}(P, L_2)$	∞	∞	∞	3	3	3
$\text{PAQUETE_EN}(P, L_3)$	∞	∞	∞	5	4	4
$\text{PAQUETE_EN}(P, C)$	∞	∞	1	1	1	1
$\text{CONECTADOS}(L_i, L_j)$ si L_i y L_j están conectados	0	0	0	0	0	0
$\text{CONECTADOS}(L_i, L_j)$ si L_i y L_j no están conectados	∞	∞	∞	∞	∞	∞

Por ejemplo, para calcular el valor de T_1^I para el hecho $\text{CAMIÓN_EN}(C, L_1)$ el algoritmo:

- Determina que las acciones que tienen a $\text{CAMIÓN_EN}(C, L_1)$ en la lista de adición son $\text{IR}(C, L_0, L_1)$, $\text{IR}(C, L_1, L_1)$, $\text{IR}(C, L_2, L_1)$ e $\text{IR}(C, L_3, L_1)$.
- Las precondiciones de $\text{IR}(C, L_0, L_1)$ son $\text{CONECTADOS}(L_0, L_1)$ y $\text{CAMIÓN_EN}(C, L_0)$, luego calcula

$$\begin{aligned} c_{\text{pre}(\text{IR}(C, L_0, L_1))} &\leftarrow T_0^I(\text{CONECTADOS}(L_0, L_1)) + T_0^I(\text{CAMIÓN_EN}(C, L_0)) \\ &= 0 + 0 \\ &= 0 \end{aligned}$$

- Las precondiciones de $\text{IR}(C, L_1, L_1)$ son $\text{CONECTADOS}(L_1, L_1)$ y $\text{CAMIÓN_EN}(C, L_1)$, luego calcula

$$\begin{aligned} c_{\text{pre}(\text{IR}(C, L_1, L_1))} &\leftarrow T_0^I(\text{CONECTADOS}(L_1, L_1)) + T_0^I(\text{CAMIÓN_EN}(C, L_1)) \\ &= \infty + \infty \\ &= \infty \end{aligned}$$

- Las precondiciones de $IR(C, L_2, L_1)$ son $CONECTADOS(L_2, L_1)$ y $CAMIÓN_EN(C, L_2)$, luego calcula

$$\begin{aligned} c_{pre(IR(C, L_2, L_1))} &\leftarrow T_0^I(CONECTADOS(L_2, L_1)) + T_0^I(CAMIÓN_EN(C, L_2)) \\ &= 0 + \infty \\ &= \infty \end{aligned}$$

- Las precondiciones de $IR(C, L_3, L_1)$ son $CONECTADOS(L_3, L_1)$ y $CAMIÓN_EN(C, L_3)$, luego calcula

$$\begin{aligned} c_{pre(IR(C, L_3, L_1))} &\leftarrow T_0^I(CONECTADOS(L_3, L_1)) + T_0^I(CAMIÓN_EN(C, L_3)) \\ &= 0 + \infty \\ &= \infty \end{aligned}$$

- Calcula $c_{CAMIÓN_EN(C, L_1)} \leftarrow \min(c_{pre(IR(C, L_0, L_1))} + c_{IR(C, L_0, L_1)}, c_{pre(IR(C, L_1, L_1))} + c_{IR(C, L_1, L_1)}, c_{pre(IR(C, L_2, L_1))} + c_{IR(C, L_2, L_1)}, c_{pre(IR(C, L_3, L_1))} + c_{IR(C, L_3, L_1)})$
 $= \min(0 + 1, \infty + 1, \infty + 1, \infty + 3)$
 $= 1$

- Establece $T_1^I(CAMIÓN_EN(C, L_1)) \leftarrow \min(T_0^I(CAMIÓN_EN(C, L_1)), c_{CAMIÓN_EN(C, L_1)})$
 $= \min(\infty, 1)$
 $= 1$

Finalmente, una vez la función T^I es estable, se tiene que

$$\begin{aligned} h^{\max}(I) &= T^I(CAMIÓN_EN(C, L_0)) + T^I(PAQUETE_EN(P, L_3)) \\ &= 0 + 4 \\ &= 4 \end{aligned}$$

h^{add} es una heurística segura y consciente del objetivo. Se verifica que $h^{\text{add}} \geq h^+$ y es, en general, no admisible. Puede, además, sobreestimar en demasía el coste real de alcanzar el objetivo, ya que ignora los subplanes compartidos entre los objetivos individuales.

Es importante entender que los algoritmos de búsqueda de un plan solución trabajan con la especificación original del problema. Es decir, los nodos del árbol de búsqueda son estados del problema y los sucesores se obtienen aplicando las acciones del problema. Es únicamente a la hora de calcular la heurística de un estado cuando se hace uso del problema relajado.